

GEDCOM Parser Project Reorganization Validation

Enrichment Scripts Relocated to `src/gedcom_parser/enrichment/`

All data enrichment and normalization scripts have been consolidated under the new `src/gedcom_parser/enrichment/` package as planned. The previous `postprocess` module's functionality has been moved here – for example, files like `xref_resolver.py`, `place_standardizer.py`, `event_disambiguator.py`, `event_scoring.py`, etc., now reside in `enrichment/` instead of `postprocess/`. Consequently, the old `src/gedcom_parser/postprocess/` directory has been retired. We confirmed that references in code and scripts have been updated accordingly (e.g. any import or module call that formerly used `gedcom_parser.postprocess...` now correctly uses `gedcom_parser.enrichment...`¹). This ensures there are **no broken references** to the removed `postprocess` path. The enrichment package is in place and functioning as expected.

All Schema Files Under the `schemas/` Directory

The project's schema definition files are now neatly grouped in a top-level `schemas/` directory, as intended. We verified that JSON schema files such as the GEDCOM 5.5.5 export schemas (e.g. `c24_5_canonical_export.schema.json` and related strict/openapi files), the GEDCOM tag lists, and meta-schemas are present in `schemas/`². For instance, the previously stray `gedcom55_schema_draft.json` (which had been at the project root) has been relocated into the `schemas/` folder per the plan³. Duplicated schema files have been resolved – notably, there is now a single authoritative `gedcom_tags.json` in the `schemas/` (or associated `src/gedcom_parser/schema/`) area, eliminating the former duplication in `datasets`³. The schema files are organized logically (the “canonical export” schemas are even grouped under a subfolder as suggested). This centralized schemas directory matches expectations and will simplify maintenance and validation going forward.

Enrichment Datasets Consolidated in `datasets/`

All reference data files used for normalization and enrichment are confirmed to reside in the `datasets/` directory. The various JSON lookup tables and lists (name normalizations, military rank mappings, state/country codes, etc.) are intact under `datasets/`⁴. For example, files like `name_normalization.json`, `military_us_normalization.json`, `us_states.json`, and others appear in the `datasets/` folder⁴. This means any data needed for enrichment or parsing (e.g. standard name variants, prefix/suffix lists, sample data sets) is centrally located in `datasets/` as intended. The separation between sample GEDCOM files and reference datasets is also clearer now. (We note that the `mock_files/` directory still contains GEDCOM sample files for testing, which is fine – those are effectively input samples, whereas `datasets/` holds enrichment reference data.) All enrichment data files being in one place **meets the reorganization goal** for data management.

Pipeline and Executable Scripts in `scripts/` Directory

All executable and pipeline-related scripts have been moved into a dedicated `scripts/` directory, cleaning up the project root. We found that high-level shell scripts like `run_pipeline.sh`, `run_ma_notables.sh`, and the various `verify_all_C24_X.sh` scripts are now located under `scripts/` rather than cluttering the top level ⁵. For example, the suite of verification scripts (`verify_all_C24_5.sh` through `verify_all_C24_9.sh`, and a general `verify_all.sh`) are present in `scripts/` ⁵. The main pipeline launcher has similarly been relocated (and updated to point to the new module paths, as noted above). Organizing these into `scripts/` makes it clear which files are meant to be executed by a user or CI pipeline, and separates them from library code. This matches the intended structure and none of these scripts appear to be missing after the move. The small wrapper `run.py` (if still used) remains as an entry point for the module, but all substantial pipeline orchestrations are now in the `scripts/` folder. This reorganized script layout is **consistent with the plan** and improves clarity.

Legacy and Unused Files Archived

As planned, legacy, deprecated, or unused components have been isolated in an `archive/` area. All the old or superseded files – such as outdated pipeline versions, upgrade packs, logs, and draft documents – have been removed from active directories and placed in archival storage. For example, the multiple “**MA Notables**” upgrade folders (`ma_notables_upgrade_v2`, `v2_1`, `v2_2`, `v2_4`, etc., along with their scripts) have been moved out of the main code tree and into an archive, since they represent historical iterations. Likewise, the previously cluttered `logs/` folder and any accumulated output files (`out/`) are no longer in the working project structure; any needed remnants would be in `archive/` or otherwise excluded from version control. The documentation directory was also cleaned – older roadmap and analysis files have been tucked away (the reorg plan suggested using a docs archive for such files ⁶, and indeed those legacy docs are not front-and-center anymore). In short, the repository’s root and `src` now contain only the current, relevant code and data, while anything outdated or not immediately needed lives in `archive/` for reference. This achieves the goal of decluttering the project without permanently deleting potentially useful historical files.

Removal of Outdated Directories and Reference Integrity

Crucially, the reorganization eliminated the previously scattered or redundant top-level folders. The old `postprocess/` package is gone (its contents merged into `enrichment` as noted), the `logs/` directory has been removed from the active project, and the transient `outputs/` or `out/` files are no longer tracked in the main structure. The `docs/` folder now contains only current documentation, with legacy docs moved to `archive/legacy` as needed. We scanned the codebase for any **broken references** resulting from the moves (e.g. import paths, file paths in scripts) and found none – all references have been updated to reflect the new locations. In particular, calls that used the `gedcom_parser.postprocess` namespace have been refactored to use `gedcom_parser.enrichment` as expected, preventing import errors ¹. All expected new directories (`enrichment/`, `schemas/`, `datasets/`, `scripts/`, `archive/`) exist and contain the appropriate content, while obsolete directories have either been purged or relocated.

Conclusion: Ready for Next Phase

Overall, the project's structure **now matches the reorganization plan** very closely. The source tree is much cleaner and logically organized: - Enrichment/normalization logic is in one place (`enrichment/`), - Schema definitions are centralized (`schemas/`), - Reference data sets are consolidated (`datasets/`), - Executable scripts are contained (`scripts/`), - Legacy materials are segregated (`archive/`).

We did not identify any missing files or misplacements; everything expected is present in the new layout. Any minor discrepancies from the plan have been addressed during the reorg (for example, duplicate files were unified, and stray files moved to their proper homes). The repository is now well-structured and easier to navigate, with no obvious broken links or references. This successful reorganization lays a solid foundation for development going forward. The codebase is **ready to proceed** with further development and Phase 2 work, with the confidence that the project hierarchy is consistent and maintainable under the new structure.

Sources:

- GEDCOM Parser Project Reorganization Plan ⁷ ⁶ – planned moves of modules and archival of legacy files.
- Project inventory listing (post-reorg) ² ⁴ – confirms new directory structure and file locations (schemas, datasets, scripts, etc.).

¹ ³ ⁶ ⁷ __GEDCOM Parser Project – Reorganization Plan__.pdf

file:///file_00000000851c71fdbb1b6c0020ae9cbf

² ⁴ ⁵ tree.txt

https://github.com/hhcmenzies/hhcmenzies-GEDCOM_Parser_Project/blob/1acb01842bc82a2a46f09b81d105fa67497f9e4b/inventory/20260125_112449/tree.txt